B5

PHẦN 1 – NHẬN DIỆN RỦI RO VÀ XỬ LÝ SỰ CỐ

1. Tình huống 1 (Email giả)

- **Tên tấn công: Phishing** + **Typosquatting** (Sai tên miền).
- **Chi tiết giả:** Sai ký tự link: `githuh-security.com` (chữ **h** thay vì chữ **b**).

2. Tình huống 2 (Tải file Crack)

- **Tên cạm bẫy: Trojan / Infostealer** (Mã độc ẩn nấp).
- **Nguy hiểm tắt Defender:** Vô hiệu hóa hàng rào bảo mật để virus chạy ngầm cướp tài khoản, cookie, SSH keys.

3. Đối chiếu Quy tắc 5 KHÔNG

- **Tình huống 1:** KHÔNG bấm link lạ, đáng ngờ.
- **Tình huống 2:** KHÔNG tải, cài phần mềm bẻ khóa (Crack).

4. Quy trình Cấp cứu 4 Bước

- **Bước 1: Ngắt mạng khẩn cấp** (rút dây LAN, tắt Wi-Fi).
- **Bước 2: Đổi mật khẩu** mọi tài khoản từ thiết bị sạch khác.
- **Bước 3: Thu hồi Access Tokens, SSH Keys** trên Git/Vercel.
- **Bước 4: Cài lại hệ điều hành sạch** (Format ổ cứng).

PHẦN 2 – CHUYỂN SANG PASSWORD MANAGER

1. Hậu quả dùng chung mật khẩu

- **Hậu quả:** Mất 1 tài khoản (Email) kéo theo mất sạch các tài khoản còn lại (Git, Vercel).
- **Tên tấn công: Credential Stuffing** (Nhồi nhét thông tin đăng nhập).

2. Cơ chế Zero-Knowledge Architecture

- Mật khẩu được mã hóa ngay tại máy người dùng trước khi lên đám mây. Máy chủ Bitwarden không giữ khóa giải mã nên hacker có hack được máy chủ cũng chỉ thấy dữ liệu vô nghĩa.

3. Bảng thông số cấu hình Generator

- **Độ dài:** ≥ 16 ký tự.
- **Chữ hoa / Chữ thường / Chữ số / Ký tự đặc biệt:** Bật (Có) toàn bộ.

PHẦN 3 – KÍCH HOẠT 2FA CHO GITHUB TỔ CHỨC

1. SMS OTP vs Authenticator App

- **Nhược điểm SMS 1:** Dễ bị tấn công cướp sim (**SIM Swap**).
- **Nhược điểm SMS 2:** Truyền tin không mã hóa, dễ bị đánh chặn hoặc nghẽn mạng.

2. Các bước kích hoạt 2FA qua Google Authenticator

- **Bước 1:** Vào **Settings** GitHub ➔ **Password and authentication**.
- **Bước 2:** Chọn **Enable two-factor authentication** ➔ Chọn **Authenticator app**.
- **Bước 3:** Mở app Google Authenticator trên điện thoại, bấm **+** và quét mã QR trên máy tính.
- **Bước 4:** Nhập mã **6 số** từ app vào GitHub để xác nhận.

3. Lưu trữ 10 mã khôi phục

- **NÊN LƯU:** In ra giấy cất két, USB mã hóa, hoặc mục bảo mật trong Password Manager.
- **KHÔNG LƯU:** Ảnh chụp điện thoại, file text trên màn hình, tin nhắn chat (Zalo, Tele).

PHẦN 4 – BẢO VỆ SECRETS TRƯỚC KHI PUSH CODE

1. Hai giá trị bị hardcode

- `sk-proj-abc123456789` và `SmartCampus@2026`.

2. Viết lại `config.js` an toàn

javascript
```
require('dotenv').config();
const API_KEY = process.env.API_KEY;
const DB_PASSWORD = process.env.DB_PASSWORD;

function connectDatabase() { return connect(DB_PASSWORD); }
```

Hãy thận trọng khi sử dụng mã.

3. Nội dung các tệp tin

Tệp `.env` (LOCAL):

text
```
API_KEY=sk-proj-abc123456789
DB_PASSWORD=SmartCampus@2026
```

Hãy thận trọng khi sử dụng mã.

Tệp `.gitignore`:

text
```
.env
node_modules/
```

Hãy thận trọng khi sử dụng mã.

Tệp `.env.example` (PUSH GIT):

text
```
API_KEY=your_api_key_here
DB_PASSWORD=your_db_password_here
```

Hãy thận trọng khi sử dụng mã.

4. Quy tắc 4 Bước Kiểm tra trước khi Push

- **Bước 1:** Check file `.gitignore` xem đã viết `.env` vào chưa.
- **Bước 2:** Chạy `git status` xác nhận file `.env` không bị đưa vào danh sách commit.
- **Bước 3:** Review lại code xem đã đổi hết sang `process.env` chưa.
- **Bước 4:** Check file `.env.example` đảm bảo không dính key thật.